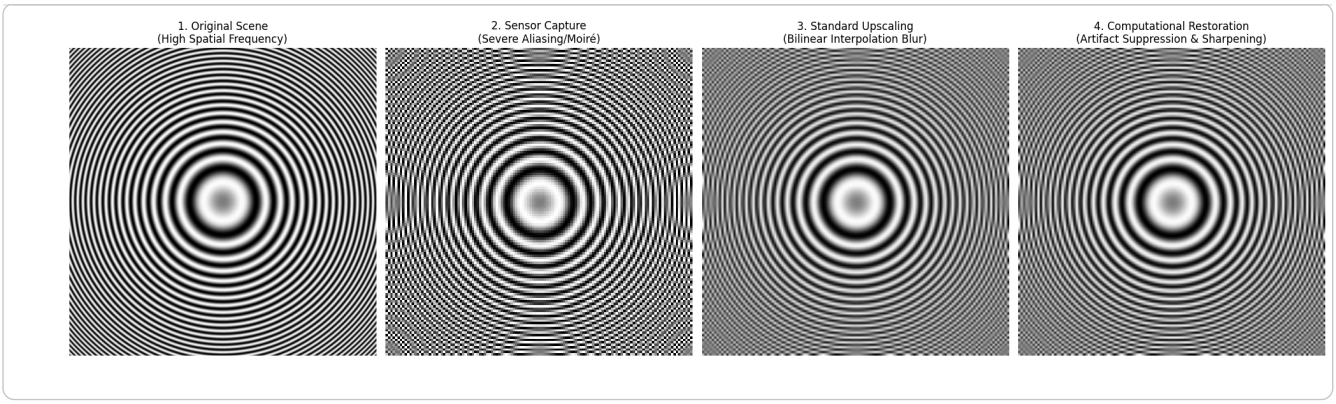
Featured examples

- Train a miniGPT language model with JAX AI Stack
- LoRA/QLoRA finetuning for LLM using Tunix
- Parameter-efficient fine-tuning of Gemma with LoRA and QLoRA
- Loading Hugging Face Transformers Checkpoints
- 8-bit Integer Quantization in Keras
- Float8 training and inference with a simple Transformer model
- Pretraining a Transformer from scratch with KerasHub
- Simple MNIST convnet
- Image classification from scratch using Keras 3
- Image Classification with KerasHub

```
import numpy as np
import cv2
import matplotlib.pyplot as plt

# 1. Generate a High-Resolution "Continuous" Scene (Zone Plate)
# This represents a scene with progressively higher spatial frequencies toward the edges.
grid_size = 1024
x = np.linspace(-10, 10, grid_size)
y = np.linspace(-10, 10, grid_size)
X, Y = np.meshgrid(x, y)
R = np.sqrt(X**2 + Y**2)
# High-resolution original signal
high_res_scene = np.sin(R**2)
# Normalize to 0-255 for image processing
high_res_scene = ((high_res_scene - high_res_scene.min()) / (high_res_scene.max() - high_res_scene.min()) * 255).astype(np.u

# 2. Simulate Improper Sampling (Undersampling)
# We sample every 8th pixel, skipping the frequencies required by the Nyquist limit.
sampling_factor = 8
undersampled_image = high_res_scene[::sampling_factor, ::sampling_factor]

# 3. Software Correction Pipeline
# Step A: Standard Bilinear Interpolation (Brings back size, but introduces "Perceived Blur")
standard_reconstruction = cv2.resize(undersampled_image, (grid_size, grid_size), interpolation=cv2.INTER_LINEAR)

# Step B: Advanced Computational Restoration
# We use a Bilateral Filter to suppress false Moiré ripples/jaggies while preserving true structural edges,
# followed by an unsharp mask to computationally reconstruct high-frequency crispness.
denoised = cv2.bilateralFilter(standard_reconstruction, d=9, sigmaColor=75, sigmaSpace=75)
kernel = np.array([[-1,-1,-1],
                   [-1, 9,-1],
                   [-1,-1,-1]])
computational_sr = cv2.filter2D(denoised, -1, kernel)

# 4. Plot and Compare the Results
plt.figure(figsize=(20, 10))

# Plot Original High-Res Scene
plt.subplot(1, 4, 1)
plt.imshow(high_res_scene, cmap='gray')
plt.title("1. Original Scene\n(High Spatial Frequency)", fontsize=12)
plt.axis('off')

# Plot Undersampled Sensor Capture
plt.subplot(1, 4, 2)
plt.imshow(undersampled_image, cmap='gray')
plt.title("2. Sensor Capture\n(Severe Aliasing/Moiré)", fontsize=12)
plt.axis('off')

# Plot Standard Upscaling (Perceived Blur)
plt.subplot(1, 4, 3)
plt.imshow(standard_reconstruction, cmap='gray')
plt.title("3. Standard Upscaling\n(Bilinear Interpolation Blur)", fontsize=12)
plt.axis('off')

# Plot Computational Correction
plt.subplot(1, 4, 4)
plt.imshow(computational_sr, cmap='gray')
plt.title("4. Computational Restoration\n(Artifact Suppression & Sharpening)", fontsize=12)
plt.axis('off')

plt.tight_layout()
plt.show()
```


1. Original Scene
(High Spatial Frequency)
2. Sensor Capture
(Severe Aliasing/Moiré)
3. Standard Upscaling
(Bilinear Interpolation Blur)
4. Computational Restoration
(Artifact Suppression & Sharpening)